

Make char16_t/char32_t string literals be UTF-16/32

Document Number: P1041R4

Date: 2019-02-18

Audience: Evolution Working Group

Reply-to: cpp@rmf.io

Changelog

- Revision 4
 - Wording changes as per Core review
- Revision 3
 - Update after inclusion of P0482 in the working draft.
- Revision 2
 - Removed mention of P1025 following its inclusion in the working draft.
- Revision 1
 - Editorial changes.

Introduction

C++11 introduced character types suitable for code units of the UTF-16 and UTF-32 encoding forms, namely `char16_t` and `char32_t`. Along with this, it also introduced new string literals whose types are arrays of those two character types, prefixed with `u` and `U`, respectively. And last but not least, it also

introduced *UTF-8 string literals*, prefixed with `u8`, with types arrays of `const char`. Of these three new string literal types, only one has a guarantee about the values that the elements of the array have; in other words, only one has a guaranteed encoding form, the *UTF-8 string literals*.

The standard text hints that the `char16_t` and `char32_t` string literals are intended to be encoded as, respectively, UTF-16 and UTF-32, but unlike it does for *UTF-8 string literals*, it never explicitly makes such a requirement.

Motivation

In defining `char16_t` string literals ([lex.string]/10), the standard makes a mention of “surrogate pairs”:

A string-literal that begins with `u`, such as `u"asdf"`, is a `char16_t` string literal. A `char16_t` string literal has type “array of n `const char16_t`”, where n is the size of the string as defined below; it is initialized with the given characters. A single *c-char* may produce more than one `char16_t` character in the form of surrogate pairs.

Further down, when defining the size of `char16_t` string literals ([lex.string]/15), there is another mention of “surrogate pairs”:

The size of a `char16_t` string literal is the total number of escape sequences, *universal-character-names*, and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [Note: The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*]

For `char32_t` string literals, the definition of their size ([lex.string]/15) essentially limits the encoding form used to one that doesn’t have more than one code unit per character:

The size of a `char32_t` or wide string literal is the total number of escape sequences, *universal-character-names*, and other characters, plus one for the terminating `U'\0'` or `L'\0'`.

Additionally, the standard constrains the range of *universal-character-names* to the range that is supported by all of the UTF encoding forms discussed here:

Within `char32_t` and `char16_t` string literals, any *universal-character-names* shall be within the range `0x0` to `0x10FFFF`.

All of these requirements, while never explicitly naming the UTF-16 or UTF-32 encoding forms, strongly imply that these are the encoding forms intended.

The C standard defines the `__STDC_UTF_16__` and `__STDC_UTF_32__` in the `<uchar.h>` header, which C++ defers to for the contents of `<cuchar>`. These macros are optional, which seems to imply some intent that the encodings for `char16_t` and `char32_t` literals be implementation-defined. However, it would be questionable for an implementation to pick any other encoding forms for these string literals: there is no well-known encoding form that uses a concept named “surrogate pair” other than UTF-16, and there is no well-known encoding form that encodes each character as a single 32-bit code unit other than UTF-32.

In practice, all implementations use UTF-16 and UTF-32 for these string literals. C++ should standardize this practice and make these requirements explicit instead of just hinting at them.

Proposal

This proposal renames “`char16_t` string literals” and “`char32_t` string literals” to “UTF-16 string literals” and “UTF-32 string literals”, to match the existing “UTF-8 string literals”, and explicitly requires the arrays created by those literals to have the values that correspond to the UTF-16 and UTF-32 (respectively) encodings of the given characters.

Technical Specifications

- Change [lex.string]/9:

A *string-literal* that begins with `u`, such as `u"asdf"`, is a ~~`char16_t` string literal~~UTF-16 string literal. A ~~`char16_t` string literal~~UTF-16 string literal has type “array of `n` `const char16_t`”, where n is the size of the string as defined below; ~~it is initialized with the given characters~~each successive element of the array has the value of the corresponding code unit of the UTF-16 encoding of the string. [Note—A single *c-char* may produce more than one `char16_t` character in the form of surrogate pairs.—end note]

- Change [lex.string]/10:

A *string-literal* that begins with `U`, such as `U"asdf"`, is a ~~`char32_t` string literal~~UTF-32 string literal. A ~~`char32_t` string literal~~UTF-32 string literal has type “array of `n` `const char32_t`”, where n is the size of the string as defined below; ~~it is initialized with the given characters~~each successive element of the array has the value of the corresponding code unit of the UTF-32 encoding of the string.

- Change [lex.ccon]/4:

A character literal that begins with the letter `u`, such as `u'x'`, is a character literal of type `char16_t`, known as a *UTF-16 character literal*. The value of a ~~`char16_t`~~UTF-16 character literal containing a single *c-char* is equal to its ISO 10646 code point value, provided that the code point value is representable with a single 16-bit code unit (that is, provided it is in the basic multi-lingual plane). If the value is not representable with a single 16-bit code unit, the program is ill-formed. A ~~`char16_t`~~UTF-16 character literal containing multiple *c-chars* is ill-formed.

- Change [lex.ccon]/5:

A character literal that begins with the letter U, such as U'x', is a character literal of type `char32_t`, known as a *UTF-32 character literal*. The value of a ~~`char32_t`~~UTF-32 character literal containing a single *c-char* is equal to its ISO 10646 code point value. A ~~`char32_t`~~UTF-32 character literal containing multiple *c-chars* is ill-formed.

- Furthermore, replace all instances of *char16_t string literal* with *UTF-16 string literal*, all instances of *char32_t string literal* with *UTF-32 string literal*, all instances of *char16_t character literal* with *UTF-16 character literal*, and all instances of *char32_t character literal* with *UTF-32 character literal*.